Hochschule Konstanz
Fakultät Informatik

Software Engineering

01 - Introduction to Scala

Prof. Dr. Marko Boger

Studiendekan AIN

Prof. Dr. Marko Boger

Raum O205

marko.boger@htwg-konstanz.de

Sprechstunde Dienstags 11:30-13:00

About this Lecture Series

- Moodle as central infrastructure
  - Register to Software Engineering in AIN
  - From here access slides, video conferences, chat channel and submit results
- Lectures are in presence
  - find recordings on Moodle
- Exercises are supported by Discord
  - https://discord.gg/AVYCEf8FTg
- The Slides are in Google Docs
- Slides are in English, communication usually is in German
- Lecture for all: Friday 9:45
- Exercise 1: Friday 11:30 or immediately after lecture
- Exercise 2: Friday 8:00

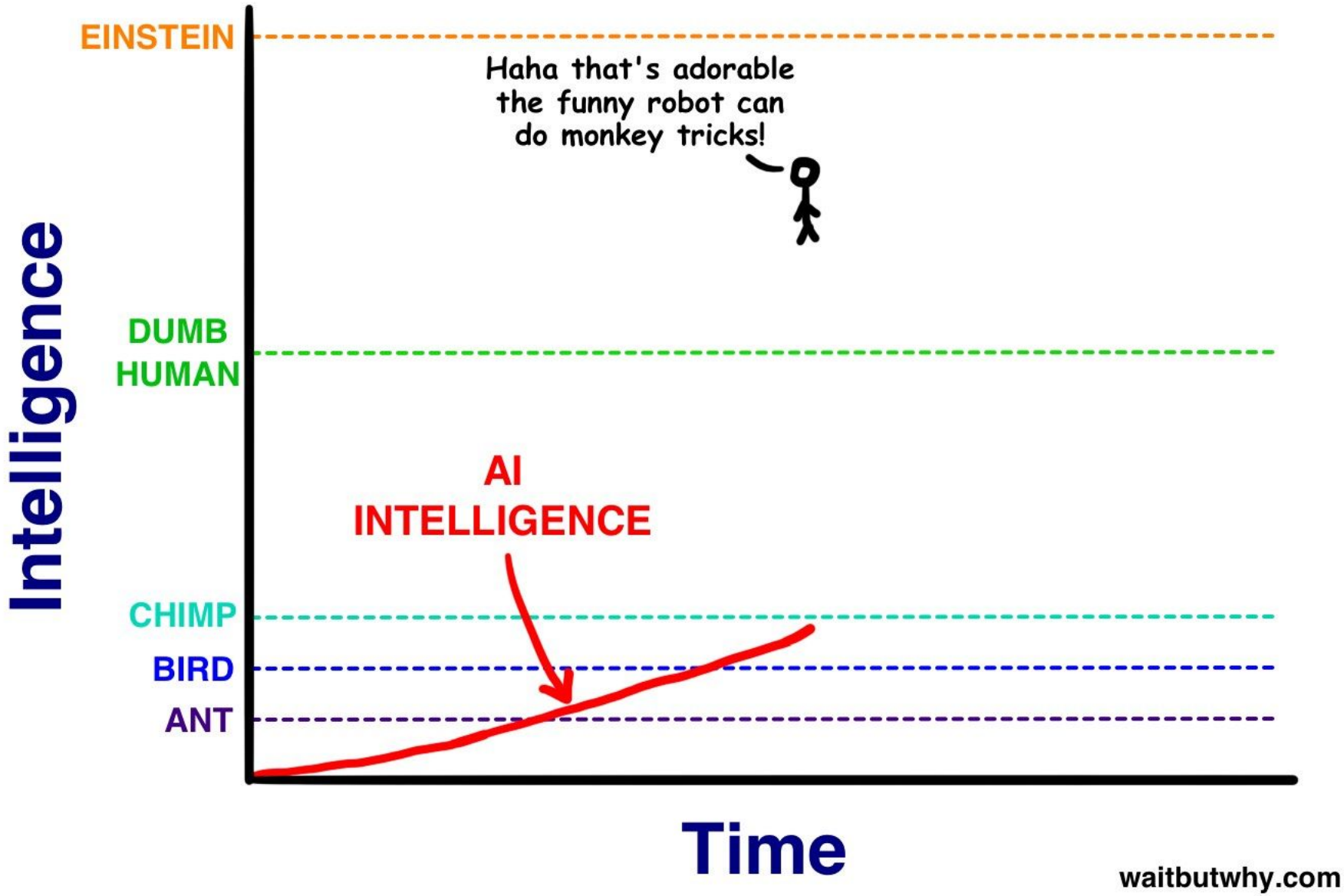
Main Goal of this Lecture

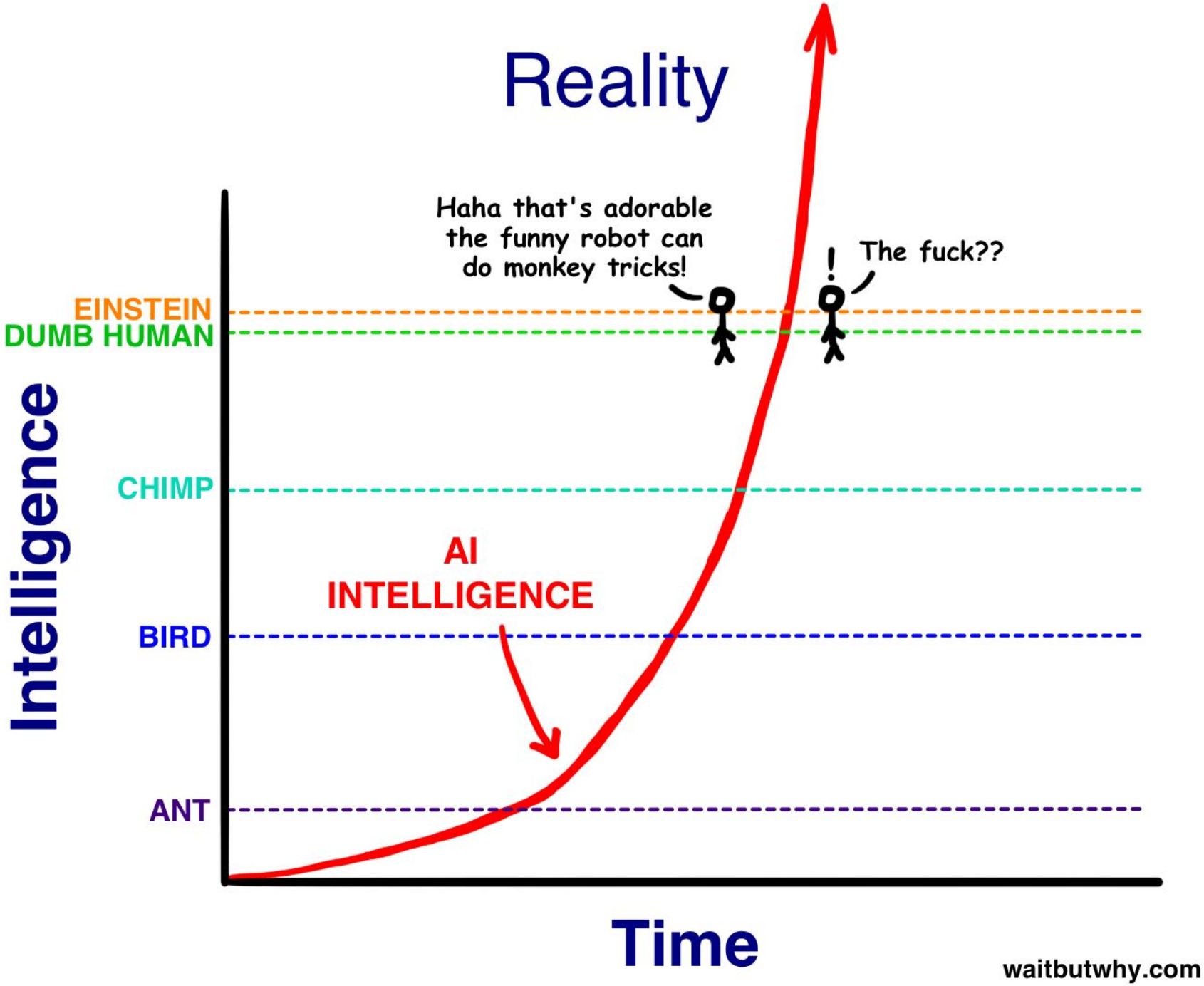
- Agility
  - The main theme of this lecture is how to achieve agility for a software project.
  - Agility is the ability to adopt changes in Business Requirements quickly.
  - This determines how we do Requirements, Planning, Development, Quality Assurance and Documentation.
  - To achieve this, we will
    - express requirements as tests
    - only have running code as artifacts
      - production and test code
    - write highly testable code
      - functional programming
    - use tests as main means to document code

Content of this Lecture

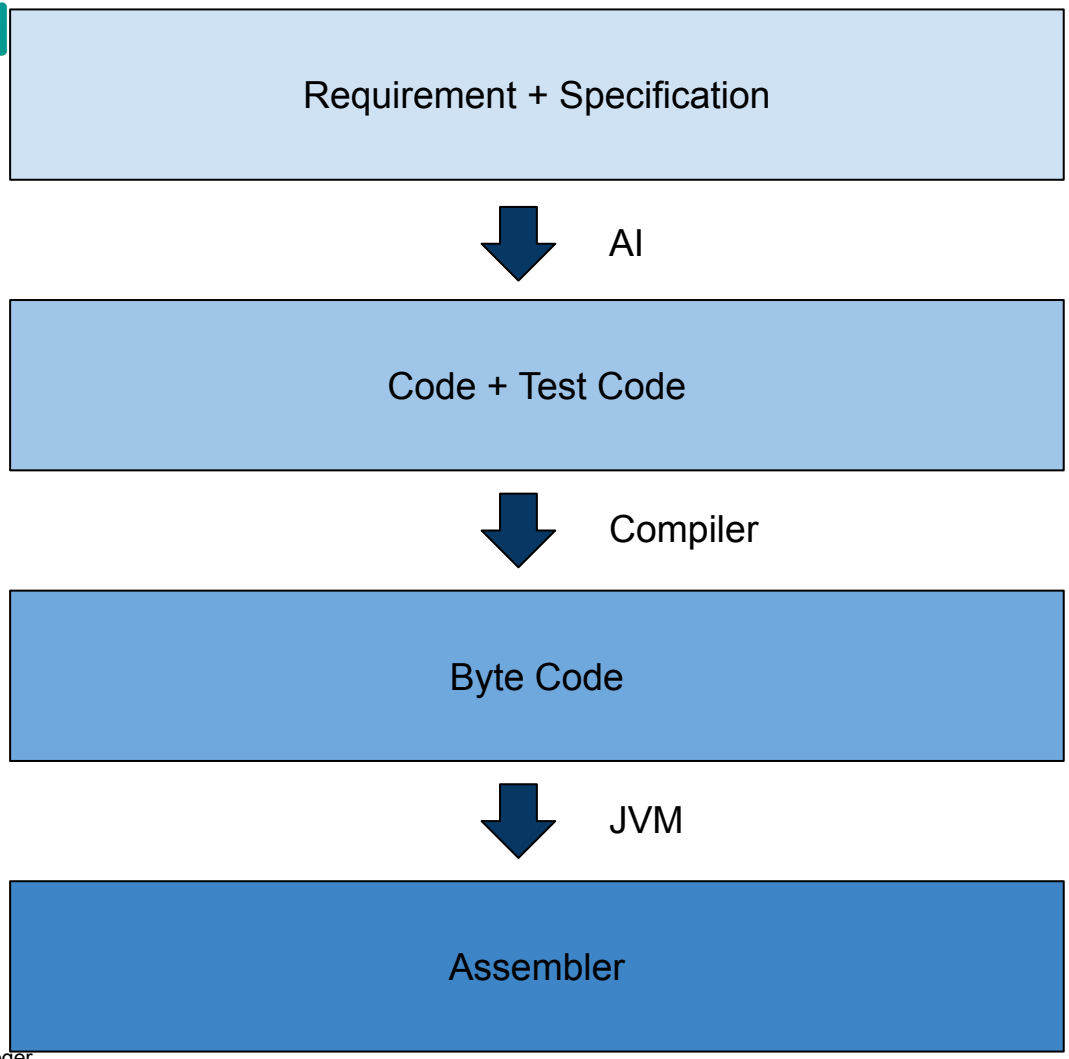
- Collaboration
  - Version control
  - Project and task management
  - Software development processes
  - AI
- Programming Techniques
  - Functional Programming
  - Tests
  - Layers
  - MVC Architecture
  - OO- and FP-Principles
  - Design Patterns
  - Components
  - Dependency Injection
- To learn all this you will develop your own software project
  - A desk top game

Our Distorted View of Intelligence

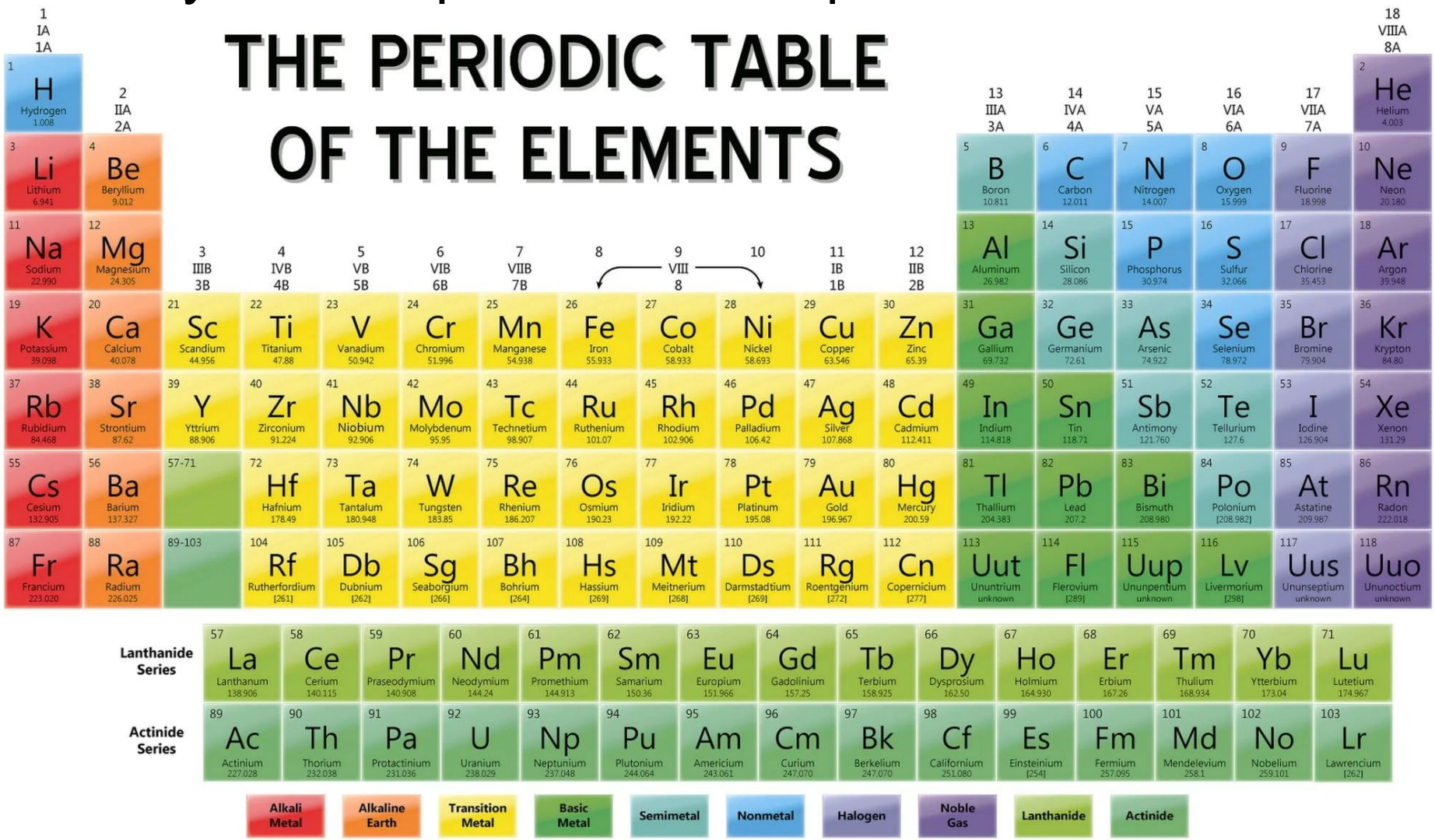



Reality
Haha that's adorable the funny robot can do monkey tricks!
The fuck??
EINSTEIN
DUMB HUMAN
CHIMP
AI INTELLIGENCE
BIRD
ANT
Intelligence
Time
waitbutwhy.com

Our Tool Stack

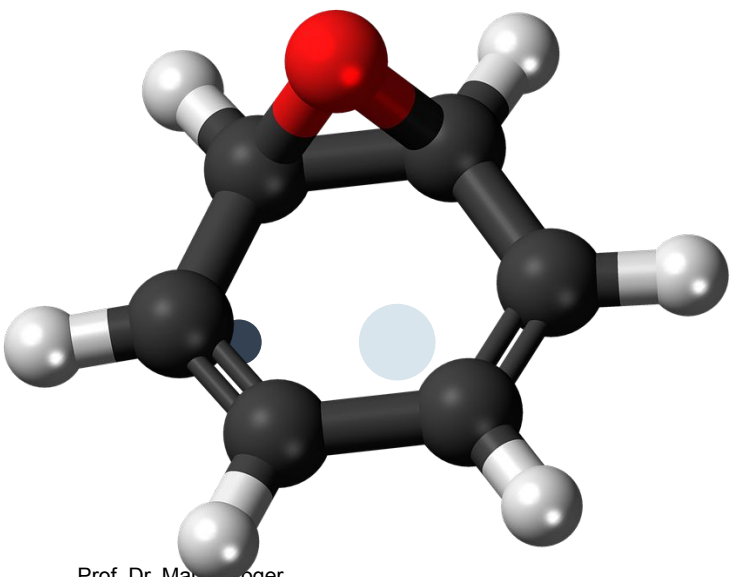


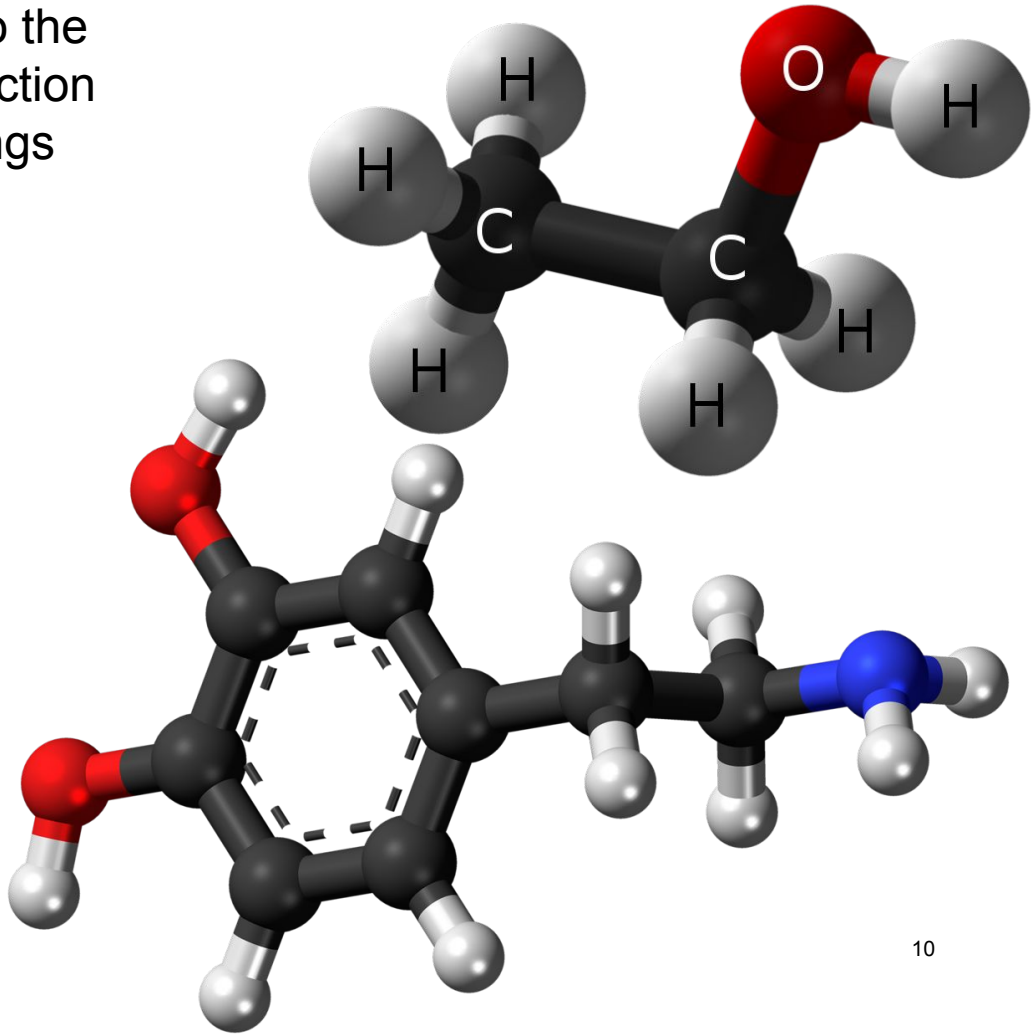
Why FP is important: a Metaphor



The importance of functional

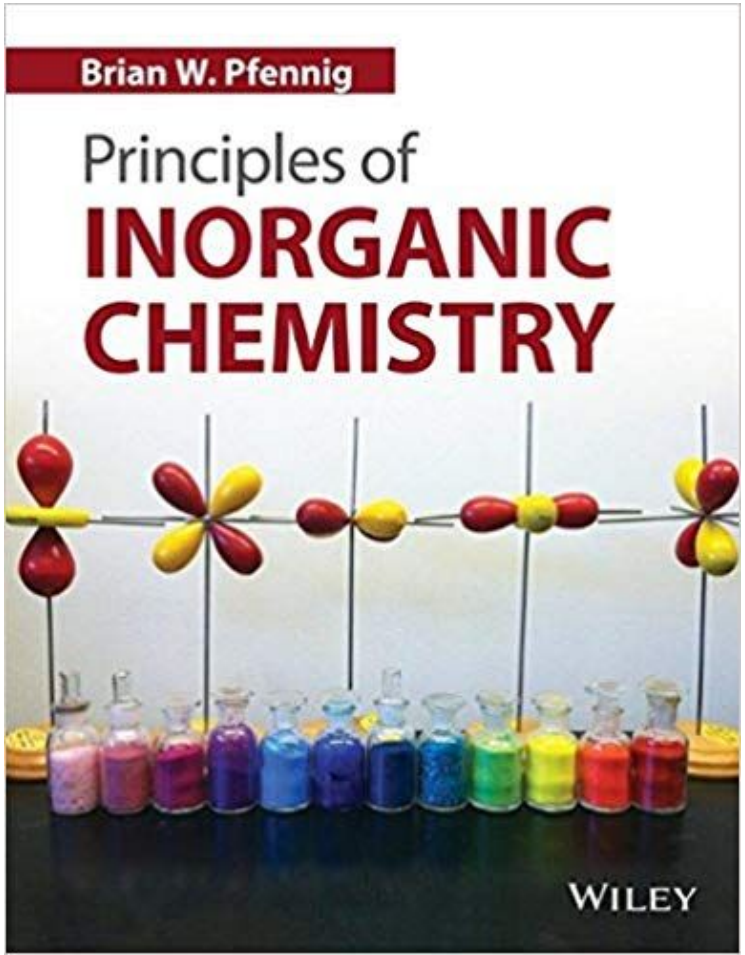
Adding functional capabilities to the programming style adds abstraction flexibility. Much like the 4 bindings of carbon create a whole new dimension for chemistry.



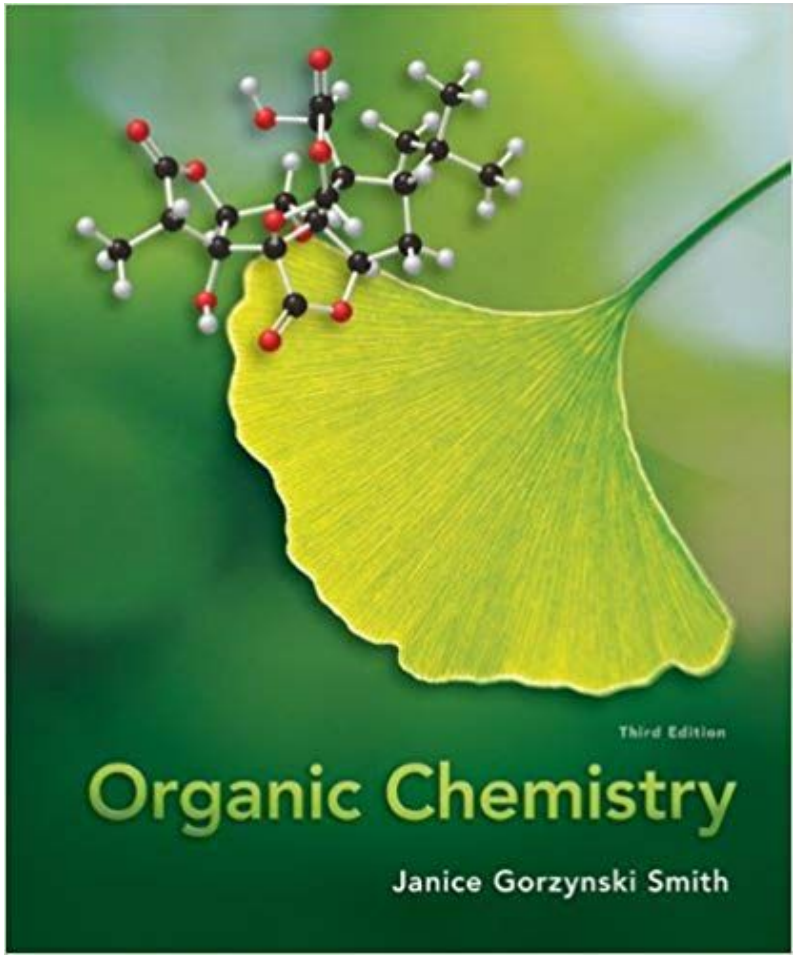


Two Schools of Chemistry

Inorganic Chemistry

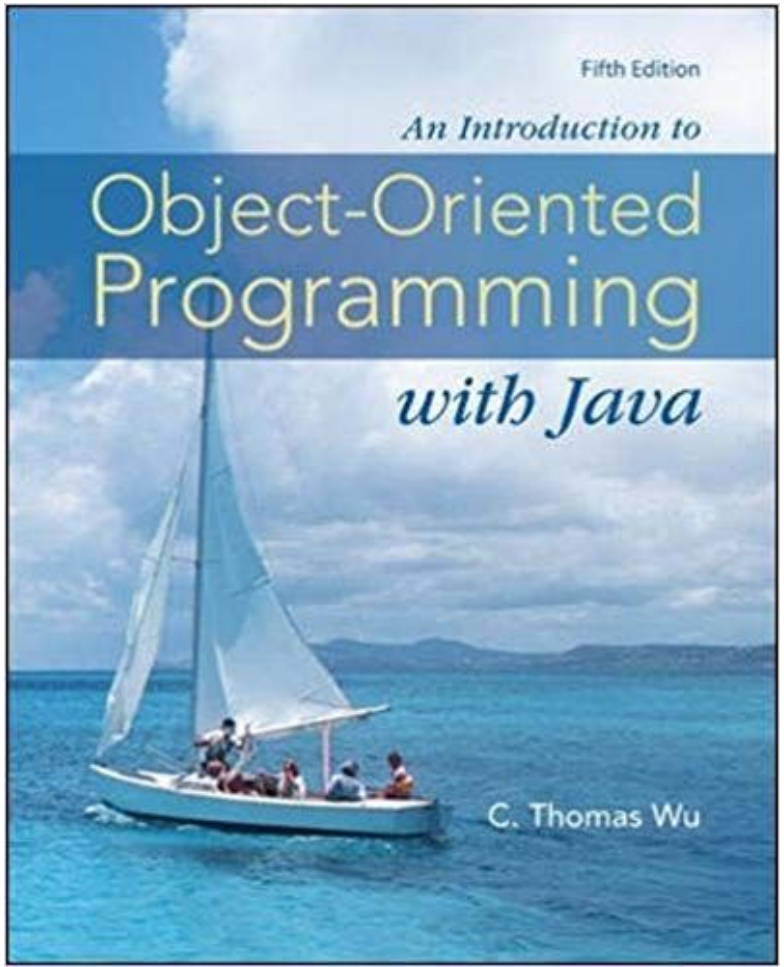


Organic Chemistry

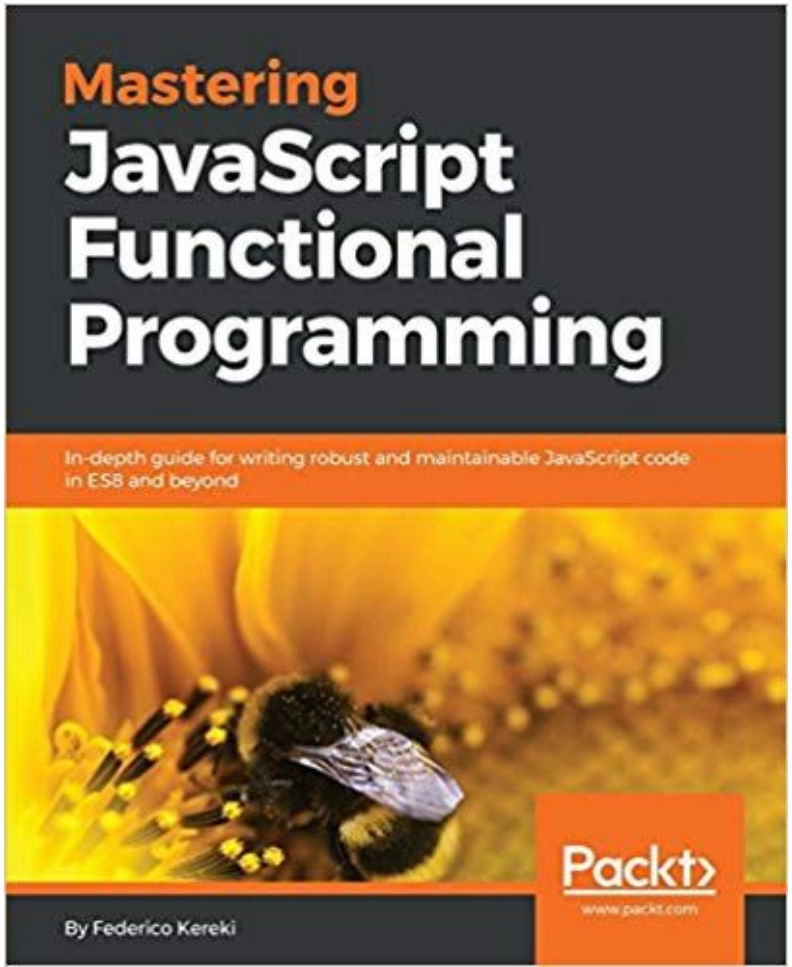


Two Schools of Programming

Object-oriented programming

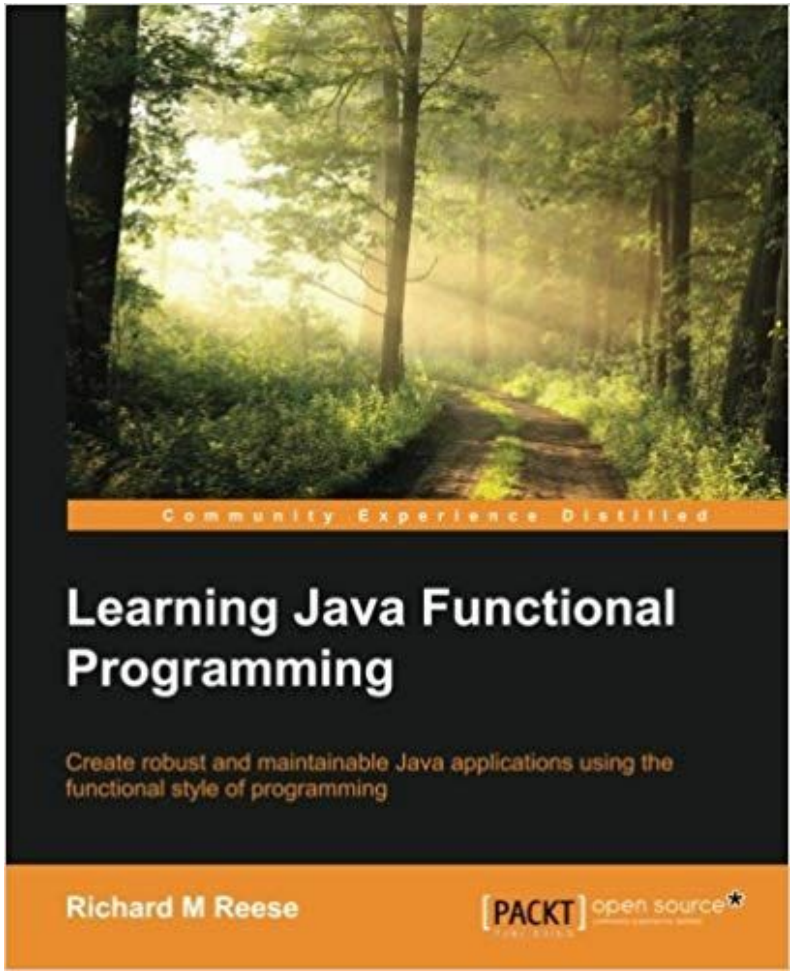


Functional Programming



These Schools are merging

Java has functional elements



Most other languages as well

So, what is functional programming?

Functions are the smallest structural element

- In OOP it is a class, an object being an instance of a class

Functions always return the same result on the same input

- In OOP objects change state and return different results

Functions can be passed as arguments

- In OOP only objects (and values) can be passed as arguments

Functional Programming avoids State Change.

- Functions that do not change state are called Pure Functions.

Die Menge aller existierenden Programmiersprachen

BASIC

Goto

Prozedurale Programmiersprachen

Pointer

Objektorientierte Programmiersprachen

Zuweisungen

Funktionale Programmiersprachen

OOP vs Functional
A very simple example

Object-Oriented

```
class PiggyBankOOP (var coins:Int = 0) {

 def insert(newCoins:Int) = coins += newCoins
 def butcher = coins
}

val pig1= new PiggyBankOOP()
pig1.insert(4)
println(pig1.butcher)
```

The pig has a changing state.

Functional

```
class PiggyBankFP (val coins:Int = 0) {

 def insert(newCoins:Int): PiggyBankFP = copy(coins + newCoins)
 def butcher = coins
}

val pig2 = new PiggyBankFP()
val pig3=pig2.insert(5)
println(pig3.butcher)
```

The pig itself never changes, it is immutable.

Instead we copy the object with a new state and manage the reference to it.

Function as parameter

Sorting

```
List(10, 5, 8, 1, 7).sorted
List("banana", "pear", "apple", "orange").sorted

def larger(x:Int, y:Int) =  x < y
List(10, 5, 8, 1, 7).sortWith(larger)
List(10, 5, 8, 1, 7).sortWith((x:Int,y:Int) => x < y)

List(10, 5, 8, 1, 7).sortWith(_ < _)
List(10, 5, 8, 1, 7).sortWith(_ > _)

List("banana", "pear", "apple", "orange").sortWith(_ < _)
List("banana", "pear", "apple", "orange").sortWith(_ > _)

List("banana", "pear", "apple", "orange").sortWith(_.length < _.length)
List("banana", "pear", "apple", "orange").sortWith(_.length > _.length)
```

Output

```
res5: List[Int] = List(1, 5, 7, 8, 10)
res6: List[String] = List(apple, banana, orange, pear)

larger: larger[](val x: Int,val y: Int) => Boolean
res7: List[Int] = List(1, 5, 7, 8, 10)
res8: List[Int] = List(1, 5, 7, 8, 10)

res9: List[Int] = List(1, 5, 7, 8, 10)
res10: List[Int] = List(10, 8, 7, 5, 1)

res11: List[String] = List(apple, banana, orange, pear)
res12: List[String] = List(pear, orange, banana, apple)

res13: List[String] = List(pear, apple, banana, orange)
res14: List[String] = List(banana, orange, apple, pear)
```

Why not Java?

Java is also incorporating functional elements into its core language since version 8. Functional programming is possible in Java, but it is not easy. For example, creating an immutable object is hard, since the default data structures are mutable. The syntax for immutable structures is longer than for mutable, state change is so common, it is hard to get away from it.

So, to become a good programmer with functional style available to you, it is easier to learn a language that makes functional programming easy and then switch back to Java and apply what you learned. If you ever go back...

Why Scala?

Desirable language features for scalable software systems are

- Static Type System
- Strong Type System
- Implicit Type System
- Pure OOP
- Functional

Scala combines these in an elegant way.

	Scala	C	Java	JavaScpt	Python	Haskell	Kotlin
Static Type System	■		■			■	■
Strong Type System	■					■	■
Implicit Type System	■				■	■	■
Pure OOP	■				■		■
Functional	■			■	■	■	■

Who invented Scala?

Martin Odersky

- Built the Java compiler for Java 1.3 and 1.4
- Built the generic type system and compiler for Java 1.5
- Now a Professor at EPFL Lausanne
- Started to create a “Java how it should be”
- Founded the company Typesafe in 2011
- Typesafe was renamed to Lightbend in 2015

Scala = a SCAlable LAnguage

- Scala is scalable in three dimensions
  - Program size
    - Interpreter, Scripts, Programs, Component Systems
  - Language extension
    - Java is a closed language
      - No operator overloading, no additional operators
    - Scala is open, it can be extended
      - Operators are methods and can be added and extended
      - Flexible syntax allows elegant extensions
      - Libraries can feel like extensions
  - Number of Processors
    - Good Concurrency System
    - Functional Programming is a good fit for concurrency

Scala is a Hybrid Language

- Scala is an Object-Oriented Language
  - Much cleaner OO than Java
  - Everything is an object
- Scala is a Functional Language
  - Functional means
    - No state change
    - Same input produces same output
    - Functions as parameters
- Scala is the first language that combines OO and FP in a very clean, straightforward fashion

JVM as Platform

Scala compiles to Java byte code

Scala compiles to .class files

Scala runs on the Java VM

Scala is fully interoperable with Java

- Call
  - Scala classes can call Java classes
  - Java classes can call almost all Scala classes
- Inherit
  - Scala classes can inherit from Java classes
  - Java classes can inherit from almost all Scala classes

Install Scala

JavaJDK 1.17, 1.21, 1.25 required

Download Scala from www.scala-lang.org

Scala can operate in an interpreted mode

- in a shell, call scala, this starts a REPL (the interpreter).
- The REPL is very good for first experiments.

```
scala> val a = List(10, 5, 8, 1, 7).sorted
a: List[Int] = List(1, 5, 7, 8, 10)

scala> val b = List("banana", "pear", "apple", "orange").sorted
b: List[String] = List(apple, banana, orange, pear)
```

Scala Commandline Interpreter

```
Markos-iMac:~ mboger$ scala
Welcome to Scala 2.12.6 (Java HotSpot(TM) 64-Bit Server VM, Java 10.0.2).
Type in expressions for evaluation. Or try :help.

scala> 17+4
res0: Int = 21

scala> def f(x:Int) = x+1
f: (x: Int)Int

scala> f(20)
res1: Int = 21

scala> val x = 42
x: Int = 42

scala> f(x)
res2: Int = 43

scala>
```

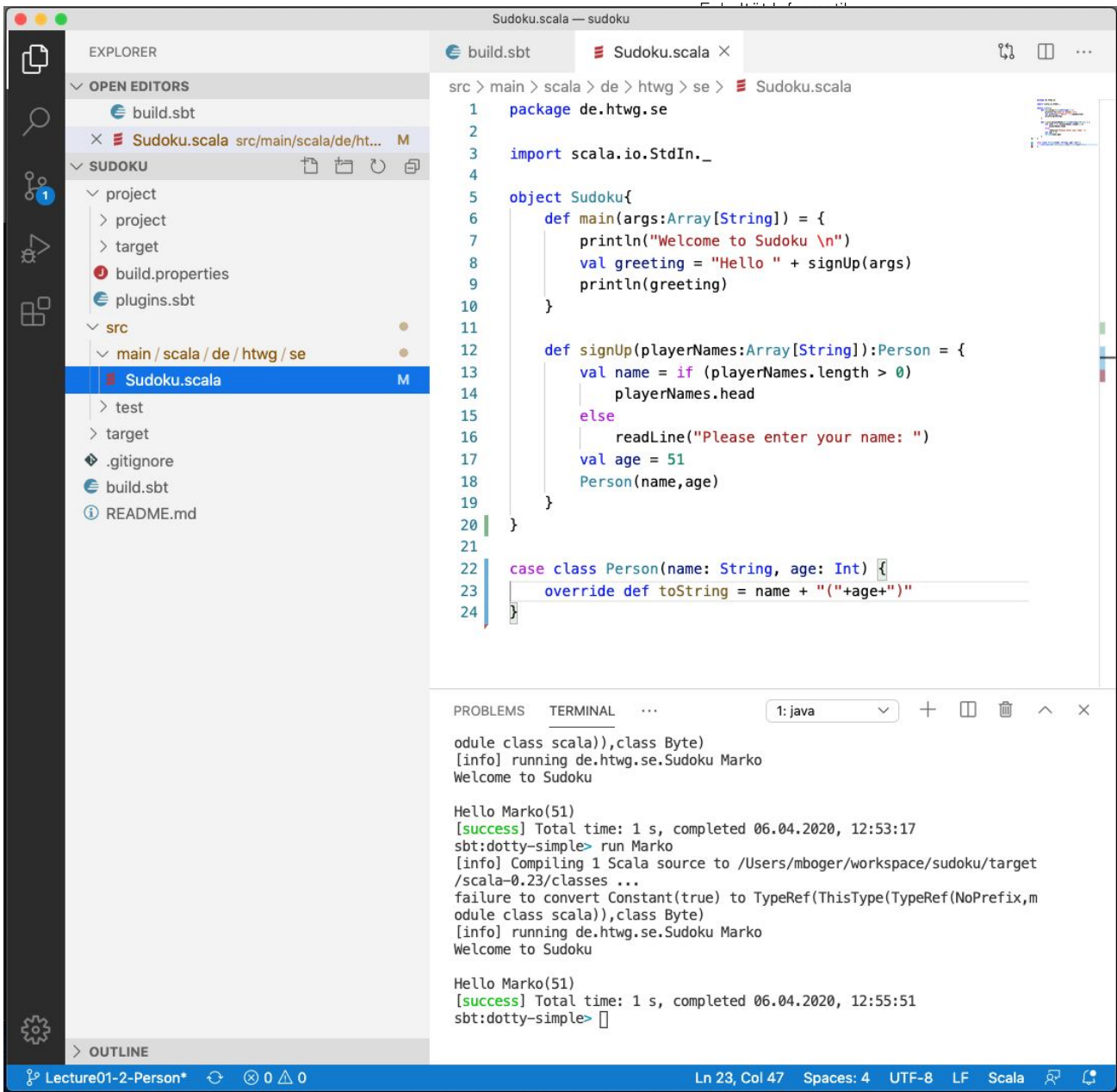
Scala in VS Code

MS VS Code is a good light weight tool to start out with.

It supports Scala 3.

To set up a project for scala, use a template:

```
sbt new
scala/scala3.g8
```



Install IDEA

A good alternative is IntelliJ IDEA.

The Community as well as the Ultimate Edition contain Scala support. I recommend the Ultimate Edition.

Scala is a plug-in. It takes quite a while after installing it before it becomes active (some minutes).

- Create a new Scala Project with sbt support
- Create a Scala Worksheet to experiment with Scala

Things to Avoid

Most of what you have learned so far remains valid.
But some things now get in the way of Agility.

Avoid at all cost (with few exceptions)

- null
  - variables should always have a value.
  - Initialise or use Option monad!
- void
  - methods should always return a value.
- var
  - variables that are not final. Variables should be immutable.
  - Use val!
- try
  - Exceptions break the controllflow.
  - Use Try monad!

First Scala Code

```scala
package de.htwg.se

import scala.io.StdIn._

object Sudoku{
   def main(args:Array[String]) = {
       println("Welcome to Sudoku \n")
       val greeting = "Hello " + signUp(args)
       println(greeting)
   }

   def signUp(playerNames:Array[String]):String = {
       if (playerNames.length > 0)
           playerNames.head
       else
           readLine("Please enter your name: ")
   }
}
```

Packages similar to Java
Import similar to Java
Objects similar to static
def defines methods
val defines values
no Semicolons
Types are behind the qualifier
Methods should always have a type
Types are often implicit
no return, last expression is returned

Basic differences of Syntax

The notation is similar to Java but

- Semicolons are usually inferred
- Types are often inferred
- Parentheses are often optional
- Almost every character is allowed in method names
  - Very few built in operators
  - What we know as operators are now methods (+, - , % …)
  - Even * can be used. As placeholder Scala uses underscore (_)
- The Dot operator for Method calls is optional (infix notation)
  - `a.add(b)` can be written as `a add b` or `a + b`
- Types are written behind the definition
  - `int i` becomes `var i:Int=0` or `var i=0`
- Declarations start with `val`, `var` or `def`

Data Types

Scala is purely Object-oriented, Java is not

- No primitive types
  - int, boolean, float etc are replaced by `Int`, `Boolean`, `Float`
- Every value is an object
  - `3`, `1.7`, `“Hello World“`, `(1,2,3)` are objects
  - Method calls like `3.toString` or `1.7.toInt` are possible
- Most types of Java are fully available
  - `java.lang.String`, `java.util.BitSet`
- Types are inferred
  - No need to say that `1+2` is an `Int`
  - No need to say that `3.toString` is a `String`

Your Software Project

You will develop your own desk-top game

- Teams of 2

Examples for topics

- Something similar to Sudoku - Only exception: Sudoku
  - Battleship („Schiffe versenken“)
  - Minesweeper
  - TicTacToe in 4*4*4 with AI
  - Chess with all possible moves but without AI
  - Connect 4 („Vier gewinnt“)
  - Uno
  - Poker
  - Back Gammon
- Not so good examples
  - Real-time games
    - Tetris
    - PacMan
  - Jump and run games
  - Egoshooter
- Totally forbidden
  - Reuse an existing game that was developed for this class
- This project will be continued in the classes
  - Web Technologies
  - Software Architecture

Requirements

- UI requirements
  - Textual UI
  - Graphical UI
    - Running in parallel
- Software Engineering requirements
  - Developed in team of 2
  - With version control Git
  - Provided on Github
  - File storage
  - Simple documentation
- Architecture Requirements
  - Strict layering
  - MVC Architecture
  - Test coverage
  - Components
  - Flexible exchange of these (DI)

First few features

- Types
  - Int, String
- Text output
  - println(“Hello World”)
- Operations
  - +
  - *
- Values
  - val y = 5
  - val z = “Hello”
- Functions
  - def f(x:Int) = x+1
  - def f(x:Int) : Int = x+1
  - @main def run = …

Scala String

Every case class in Scala has a String representation `toString`

`toString` can be overridden

```
case class Person(name: String, age: Int) {
   override def toString = name + "("+age+")"
}
```

Operation + on String works on all Value type and case classes

Standard operations

- `S.length` is the length of the string in characters;
- `S.substring(i)` returns the part of the string starting at index `i`.
- `S.substring(i,j)` returns the part of the string starting at index `i` and going up to index `j-1`. You can write `S.slice(i,j)` instead.
- `S.contains(T)` returns `true` if `T` is a substring of `S`;
- `S.indexOf(T)` returns the index of the first occurrence of the substring `T` in `S` (or -1);
- `S.indexOf(T, i)` returns the index of the first occurrence after index *i* of the substring `T` in `S` (or -1);
- `S.toLowerCase` and `S.toUpperCase` return a copy of the string with all characters converted to lower or upper case;
- `S.capitalize` returns a new string with the first letter only converted to upper case;
- `S.reverse` returns the string backwards;
- `S.isEmpty` is the same as `S.length == 0`;
- `S.nonEmpty` is the same as `S.length != 0`;
- `S.startsWith(T)` returns `true` if `S` starts with `T`;
- `S.endsWith(T)` returns `true` if `S` ends with `T`;
- `S.replace(c1, c2)` returns a new string with all characters `c1` replaced by `c2`;
- `S.replace(T1, T2)` returns a new string with all occurrences of the substring `T1` replaced by `T2`;
- `S.trim` returns a copy of the string with white space at both ends removed;
- `S.format(arguments)` returns a string where the percent-placeholders in `S` have been replaced by the arguments (see example below);
- `S.split(T)` splits the string into pieces and returns an array with the pieces. `T` is a regular expression (not explained here). To split around white space, use `S.split("s+")`.

Printf

Printf replaces placeholders (%) with a format (f,d, s)

```
def main(args: Array[String]) {
      var n1 = 78.99
      var n2 = 49
      var s1 = "Hello, World!"
      var f1 = printf("The value of the float variable is " +
                      "%f, while the value of the integer " +
                      "variable is %d, and the string " +
                      "is %s", n1, n2, s1)
      println(f1)
   }
```

String Interpolation

String interpolation goes even further, it replaces a variable name preceded by $ with its value.

The s interpolation simply replaces variables

```
scala> println(s"$hannah.name has a score of $hannah.score")
  Student Hannah has a score of Student 95
```

The f interpolation also formats them

```
scala> println(f"$name is $age years old, and weighs $weight%.0f
pounds.")
  Fred is 33 years old, and weighs 200 pounds.
```

Multiline String

With triple quotes you can create multiline strings

```
val foo = """This is
    a multiline
      String"""
```

Pipe allows margins, stripMargin removes them again

```
val speech = """Four score and
               |seven years ago""".stripMargin
```

\n represents a line break

```
val speech = """Four score and
              |seven years ago
              |our fathers
  """.stripMargin.replaceAll("\n", " ")
```

Task 1.1: Create a game project

Install Scala

Install VS Code

Create a Scala Project using Giterate (g8)

```
sbt new scala/scala3.g8
```

Generate the String output of a playing field on the Console

Worksheets

A worksheet is a tool that provides instant feedback for your Scala code.

Code and test code are mixed in one file.

The file can be executed and evaluated.

This is the fastest way to get feedback on your code.

Develop your data structures and classes in a worksheet.

Only after you have learned enough, copy the code to a proper class file.

Use the test code to establish tests.

Example Worksheet

```
case class Cell(value:Int) {
 def isSet:Boolean = value != 0
}
```

```
defined class Cell
```

```
val cell1= Cell(2)
cell1.isSet
```

```
cell1: Cell = Cell(2)
res0: Boolean = true
```

```
val cell2= Cell(0)
cell2.isSet
```

```
cell2: Cell = Cell(0)
res1: Boolean = false
```

```
case class Field(cells: Array[Cell])
```

```
defined class Field
```

```
val field1 = Field(Array.ofDim[Cell](1))
field1.cells(0)=cell1
```

```
field1: Field = Field([Lde.htwg.se.sudoku.model.A$...)
field1.cells(0): Cell = Cell(2)
```

```
case class House(cells:Vector[Cell])
```

```
defined class House
```

```
val house = House(Vector(cell1,cell2))
```

```
house: House = House(Vector(Cell(2), Cell(0)))
```

```
house.cells(0).value
house.cells(0).isSet
```

```
res2: Int = 2
res3: Boolean = true
```

Worksheet in IDEA

ModelWorksheet.sc

Sudoku.scala | ModelWorksheet.sc

```scala
case class Cell(value:Int) {
  def isSet:Boolean = value != 0
}

val cell1= Cell(2)
cell1.isSet

val cell2= Cell(0)
cell2.isSet

case class Field(cells: Array[Cell])

val field1 = Field(Array.ofDim[Cell]( n1= 1))
field1.cells(0)=cell1

case class House(cells:Vector[Cell])

val house = House(Vector(cell1,cell2))

house.cells(0).value
house.cells(0).isSet

case class Matrix[T] (rows:Vector[Vector[T]]) {
  def this(size:Int, filling:T) = this(Vector.tabulate(size, size){(row, col) => filling})
}

val matrix = Matrix(Vector(Vector(cell1,cell2)))

matrix.rows(0)(0).value

case class Grid(cells:Matrix[Cell]) {
  def this(size:Int) = this(new Matrix[Cell](size, Cell(0)))
}

val grid1 = new Grid( size= 4)

grid1.cells.rows(0)(0).value
```

```scala
defined class Cell



cell1: Cell = Cell(2)
res0: Boolean = true

cell2: Cell = Cell(0)
res1: Boolean = false

defined class Field

field1: Field = Field([Lde.htwg.se.sudoku.model.A$A7$A$A7$Cell;@4c19fe5b)
field1.cells(0): Cell = Cell(2)

defined class House

house: House = House(Vector(Cell(2), Cell(0)))

res2: Int = 2
res3: Boolean = true

defined class Matrix



matrix: Matrix[Cell] = Matrix(Vector(Vector(Cell(2), Cell(0))))

res4: Int = 2

defined class Grid



grid1: Grid = Grid(Matrix(Vector(Vector(Cell(0), Cell(0), Cell(0), Cell(0)), Vector(Cell(0), Cell(0), Cell(0), Cel

res5: Int = 0
```

Task 1.2: Create a Worksheet

Create a worksheet in your project

Create some basic data structures for your game in the worksheet

Access your data

Try to improve your implementation in small increments until your data and its access feel really good. Constantly test this.

In VS Code Worksheets have the ending worksheet.sc

Difficulties with VS Code

On Windows: If Unicode is not displayed, enter `chcp 65001` in console